- Scoped

For each lifetime, there is an extension method that starts with the word `Add`, followed by the name of the lifetime. For example, you have used the `AddTransient` method to register the `ICityService` with the transient lifetime.

SINGLETON SERVICES

Services registered with the `AddSingleton` method are instantiated as singletons when the service is first requested and retained for the duration of the container's lifetime, which is typically the same as the running application. As its name suggests, only one instance of a singleton can exist. It is reused for all requests. The vast majority of framework services—model binding, routing, logging, and so on—are registered as singletons. They all share the same characteristics in that they don't have any state and are thread safe, which means the same instance can be used across multiple threads; this may be required to process concurrent requests. The same characteristics must also apply to any dependencies the singleton services rely on that are instantiated along with the service.

I will briefly digress from the main direction of our current application to explore how this works with a simple demonstration. You will create a service that exposes a value that is set in its constructor, and then register that service as a singleton. You will use a GUID for the value because it is almost certain to be different each time it is generated. Then you will render that value to the browser. You will note that the value doesn't change when you refresh the page. Add a new C# class to the Services folder named `LifetimeDemoService` with the following code.

Listing 7.6 The `LifetimeDemoService` class